

Modelado y Programación, Proyecto 1: Chat

Mayté García Franco

Septiembre 2026

1 Introducción

2 Análisis

2.1 Investigación de Cliente-Servidor

El servidor espera la petición del cliente por un puerto específico. Cuando el cliente realice la petición, el servidor abrirá una conexión. Esta petición se hace por un puerto previamente establecido y, en caso de ser aceptada, se crea una conexión donde el servidor y el cliente pueden intercambiar información.

2.1.1 Puertos

Existen 65535 puertos disponibles para cada dirección IP. Sin embargo, hay algunos que están prohibidos usar ya que están designados a servicios del sistema. Estos serían los puertos del 0 al 1023 ya que están reservados para procesos del sistema. Los puertos del 1024 al 49151 pueden usarse libremente al igual que los puertos del 49152 al 65535, solo que estos últimos son de tipo temporal.

2.1.2 Sockets

Un socket nos va a permitir la comunicación bidireccional entre procesos a través de la red. Veamos como funcionan los sockets de acuerdo al modelo cliente-servidor:

Servidor:

- Creación del socket: el servidor crea un socket y lo asocia a una dirección IP y un número de puerto específico.
- Escucha: el socket del servidor espera conexiones entrantes de clientes.
- Aceptación de conexiones: cuando un cliente intenta conectarse, el servidor puede aceptar la conexión.

Cliente:

- Creación del socket: el cliente crea un socket sin asociarlo a una dirección específica.
- Conexión al servidor: el socket del cliente intenta conectarse al socket del servidor utilizando la dirección IP y el puerto del servidor.
- Comunicación: ya establecida la conexión, hay una comunicación bilateral.

2.2 División del problema

Ya con un conocimiento básico de cómo debería funcionar el Cliente y el Servidor junto con el protocolo, lo siguiente es realizar el famoso “divide y vencerás”.

Para el servidor tendríamos lo siguiente:

- Usuarios
 - Identifica
 - Cambia status
 - Desconecta
- Mensajes
 - Recibe mensaje
 - Envía mensaje
- Salas
 - Crear sala
 - Invitar usuario
 - Agregar usuario
 - Eliminar usuario
 - Lista de usuarios

Para el cliente tendríamos lo siguiente:

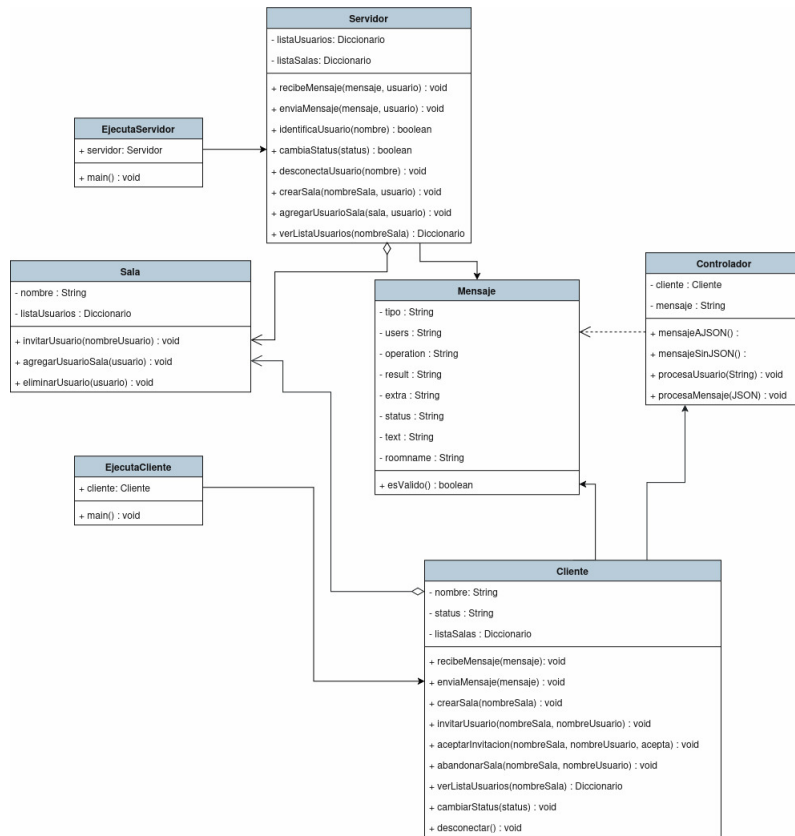
- Mensajes
 - Recibe mensaje
 - Manda mensaje
- Salas
 - Crear sala
 - Invitar usuario
 - Aceptar invitación
 - Abandonar sala
 - Ver lista de usuarios
- Cambiar status
- Abandonar servidor

3 Modelado

3.1 Paradigma

Se optó por usar el paradigma de Orientado a Objetos. Este paradigma nos ayudará a tener claras las funciones que llevan a cabo tanto el cliente como el servidor, logrando que lo que haga cada uno no se mezcle.

3.2 Diseño de clases



3.3 Lenguajes de Programación y Sistemas de Construcción

Para este proyecto se decidió programar el servidor en **Go** debido a la gran ventaja que nos brindan las Gorutines. De igual forma, su recolector de basura ayudará a facilitar el manejo del servidor y los recursos que pueda utilizar.

Del lado del cliente se decidió que se programará en **C#** ya que tiene un recolector de basura que trabaja de forma más “entendible” o, desde mi punto de vista, interesante a comparación de C o C++. Otro punto a favor de C# es

que se caracteriza por ser orientado a objetos, por lo que me facilitará un poco más la implementación de este.

En cuanto al sistema de construcción, se usará Taskfile.

4 Bibliografía

https://repositorio-uapa.cuaed.unam.mx/repositorio/moodle/pluginfile.php/3064/mod_resource/content/1/Unidades/Unidad%201/Sockets/index.html

<https://onmex.mx/tecnologia-y-desarrollo/que-es-un-socket/>